



Winning Space Race with Data Science

Sara Qureishi

22 January 2025



Outline

1. Executive Summary
2. Introduction
3. Methodology
4. Results
5. Conclusion

Executive Summary

- **Summary of methodologies**

- 1. Data Collection:**

- Used SpaceX's API to retrieve Falcon 9 launch data.
- Performed web scraping from the Falcon 9 Launch Wiki page using BeautifulSoup.
- Filtered dataset to include only Falcon 9 launches.
- Addressed missing values by replacing PayloadMass None values with the mean.

- 2. Data Wrangling:**

- Calculated key metrics like the number of launches per site and mission outcomes per orbit.
- Extracted unique orbit types and created a new label for landing outcomes.
- Analyzed launch success rate for sites like CCAFS SLC 40.

Executive Summary

3. EDA:

- Conducted SQL queries to explore launch data, calculate payload mass totals, and mission outcome counts by launch site.
- Visualized relationships between parameters like FlightNumber, PayloadMass, and Orbit.
- Examined launch success trends across years.

4. Interactive Visual Analytics:

- Developed a Folium map to visualize launch sites, marking successful and failed launches.
- Implemented a Plotly Dash dashboard with drop-down inputs, range sliders, and callback functions for dynamic charts (e.g., success pie chart and payload-outcome scatter plot).

Executive Summary

5. Predictive Analysis:

- Standardized data and split it into train and test sets.
- Evaluated machine learning models (SVM, Decision Trees, Logistic Regression) and tuned hyperparameters.
- Selected the best-performing model based on test data accuracy.

Executive Summary

- **Summary of all results**
 1. Exploratory Data Analysis result
 2. Interactive analytics in screenshots
 3. Predictive Analytics result

Introduction

- **Project background and context**

SpaceX's Falcon 9 is a game-changer in the aerospace industry due to its reusable first-stage rockets, reducing launch costs significantly. While SpaceX advertises launches for \$62 million, competitors' costs exceed \$165 million per launch. These cost reductions hinge on the success of landing and reusing the first stage.

The aim of this project is to predict whether the Falcon 9 first stage will land successfully. Understanding landing success not only influences cost analysis but also aids alternative companies in strategically bidding against SpaceX for launches. This capstone uses historical launch data and predictive modeling to uncover patterns influencing landing outcomes. By leveraging machine learning, interactive dashboards, and geospatial analysis, the project provides a comprehensive toolkit for understanding Falcon 9's operations.

Introduction

- **Problems you want to find answers**

1. What factors influence the success of Falcon 9's first-stage landings?
2. How do launch site locations, payload mass, and orbit type affect landing outcomes?
3. Can we build a predictive model to forecast the likelihood of a successful landing?

Section 1

Methodology

Methodology

Executive Summary

- Data collection methodology:
 - Data was collected using SpaceX API and web scraping from Wikipedia.
- Perform data wrangling
 - One-hot encoding was applied to categorical features
- Perform exploratory data analysis (EDA) using visualization and SQL
- Perform interactive visual analytics using Folium and Plotly Dash
- Perform predictive analysis using classification models
 - How to build, tune, evaluate classification models

Data Collection

Describe the Process

- Collected datasets via SpaceX API and web scraping
- Organized and integrated data into a structured format for analysis

Task 1: Request and parse the SpaceX launch data using the GET request

To make the requested JSON results more consistent, we will use the following static response object for this project:

```
[9]: static_json_url='https://cf-courses-data.s3.us.cloud-object-storage.appdomain.cloud/IBM-DS0321EN-SkillsNe'
```

We should see that the request was successful with the 200 status response code

```
[10]: response=requests.get(static_json_url)
```

```
[11]: response.status_code
```

```
[11]: 200
```

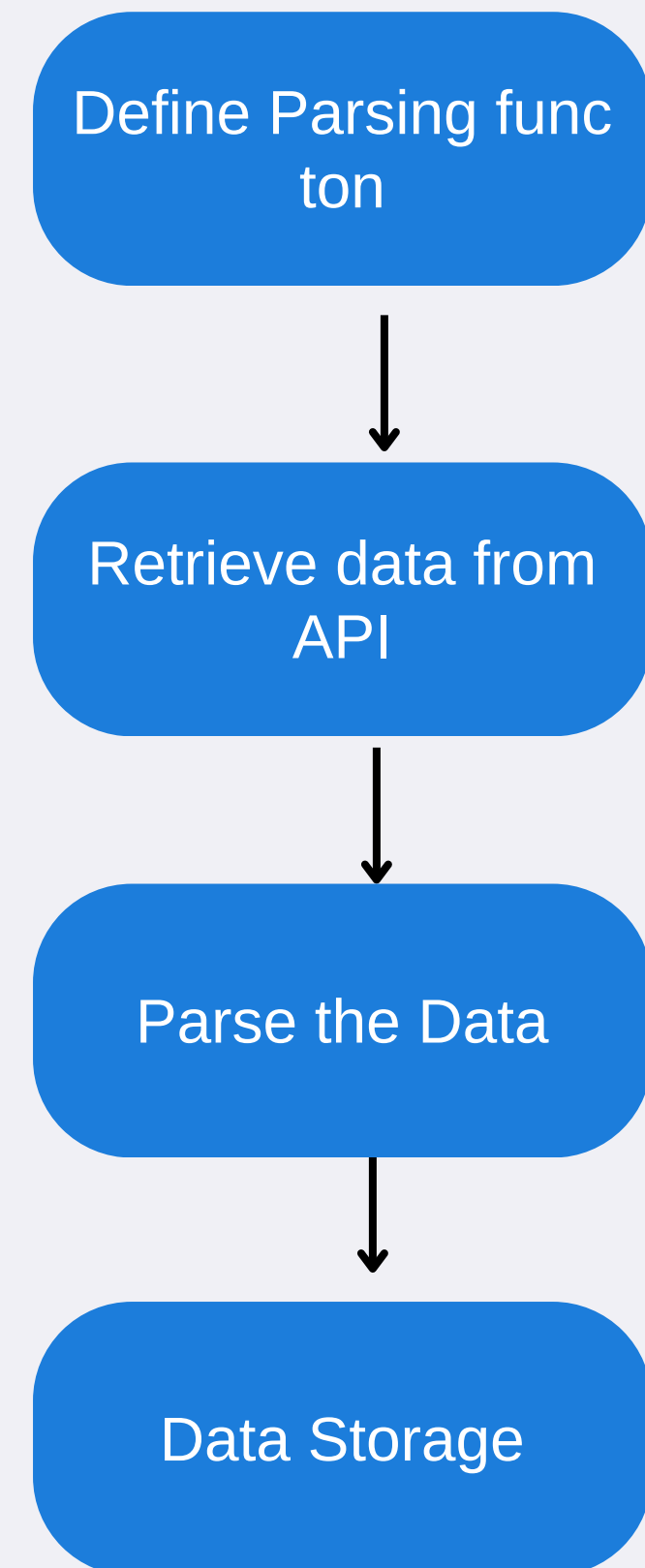
Now we decode the response content as a Json using `.json()` and turn it into a Pandas dataframe using `.json_normalize()`

```
[12]: # Use json_normalize meethod to convert the json result into a dataframe
```

Data Collection – SpaceX API

The link to the notebook is:

<https://github.com/saraquireishi/AppliedDataScienceCapstone/blob/main/MODULE-1/jupyter-labs-spacex-data-collection-api.ipynb>



Data Collection - Scraping

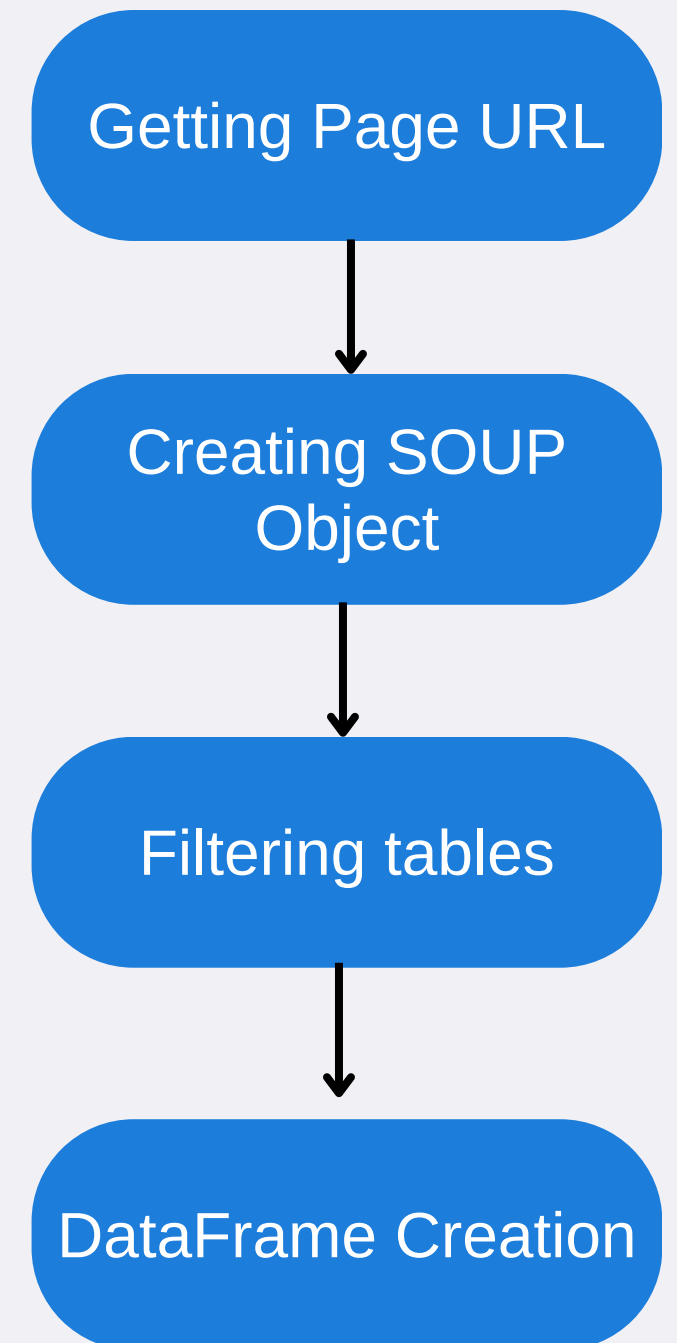
Process Highlights

Scraped Falcon 9 launch records table from Wikipedia

Parsed HTML table into Pandas DataFrame

The link to the notebook is:

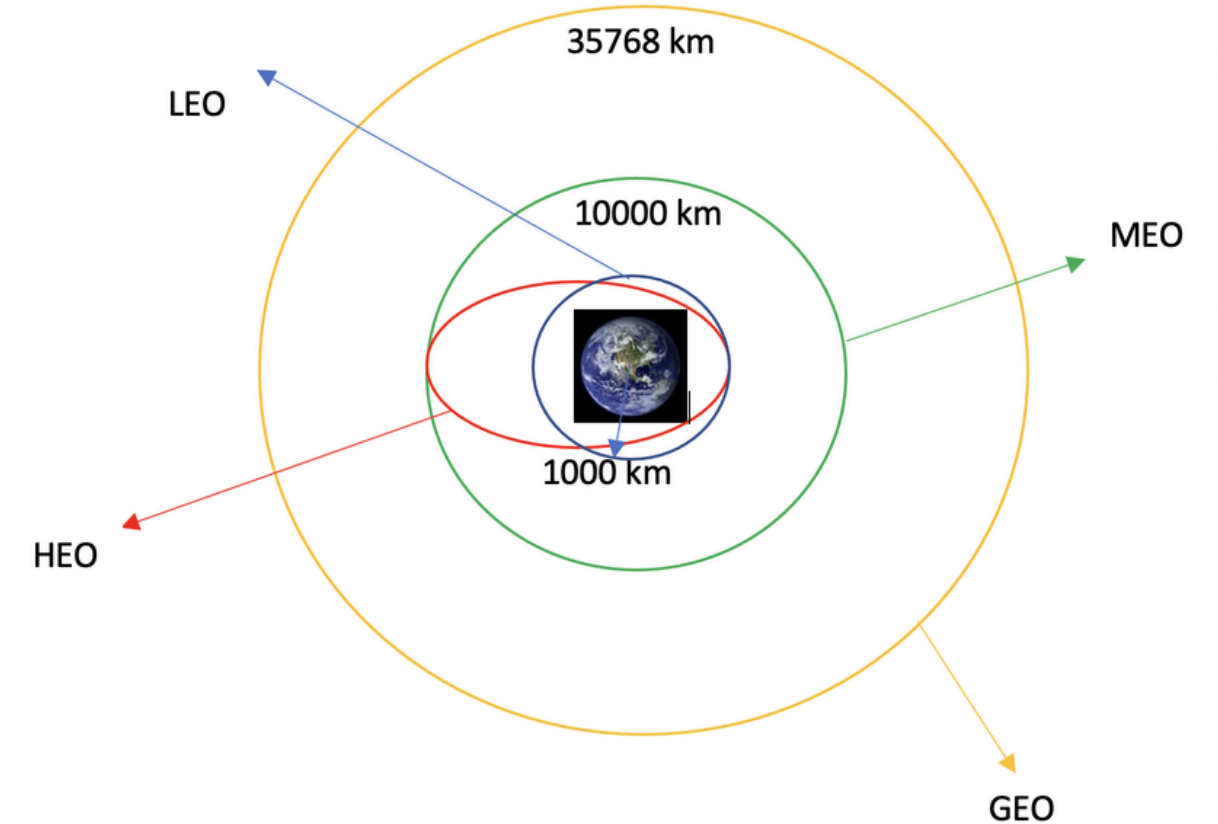
<https://github.com/saraquireishi/AppliedDataScienceCapstone/blob/main/MODULE-1/jupyter-labs-webscraping.ipynb>



Data Wrangling

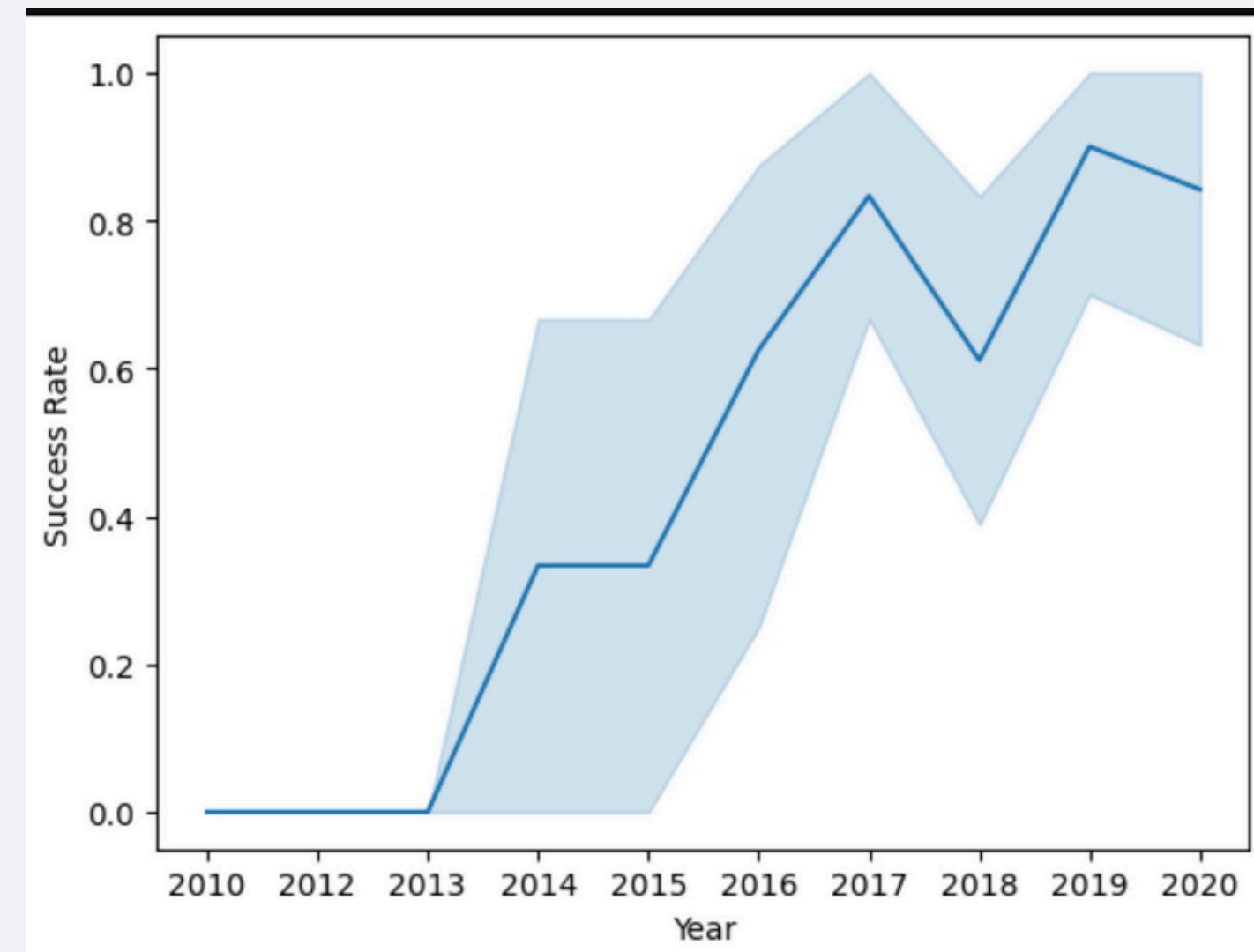
- We performed exploratory data analysis and determined the training labels.
- We calculated the number of launches at each site, and the number and occurrence of each orbits.
- We created landing outcome label from outcome column and exported the results to CSV.
- The link to notebook is:

<https://github.com/saraquireishi/AppliedDataScienceCapstone/blob/main/MODULE-1/labs-jupyter-spacex-Data%20wrangling.ipynb>



EDA with Data Visualization

- We explored the data by visualizing the relationship between flight number and launch Site, payload and launch site, success rate of each orbit type, flight number and orbit type, the launch success yearly trend.
- The Link to Notebook: [Click Here](#)



EDA with SQL

- We loaded the SpaceX dataset into a PostgreSQL database without leaving the jupyter notebook.
- We applied EDA with SQL to get insight from the data. We wrote queries to find out for instance:
 1. The names of unique launch sites in the space mission.
 2. The total payload mass carried by boosters launched by NASA (CRS)
 3. The average payload mass carried by booster version F9 v1.1
 4. The total number of successful and failure mission outcomes
 5. The failed landing outcomes in drone ship, their booster version and launch site names.
- Notebook: [Link](#)

Build an Interactive Map with Folium

- We marked all launch sites, and added map objects such as markers, circles, lines to mark the success or failure of launches for each site on the folium map.
- We assigned the feature launch outcomes (failure or success) to class 0 and 1 .i.e., 0 for failure, and 1 for success.
- Using the color-labeled marker clusters, we identified which launch sites have relatively high success rate.
- We calculated the distances between a launch site to its proximities. We answered some question for instance:
 1. Are launch sites near railways, highways and coastlines.
 2. Do launch sites keep certain distance away from cities.

Notebook: [Link](#)

Build a Dashboard with Plotly Dash

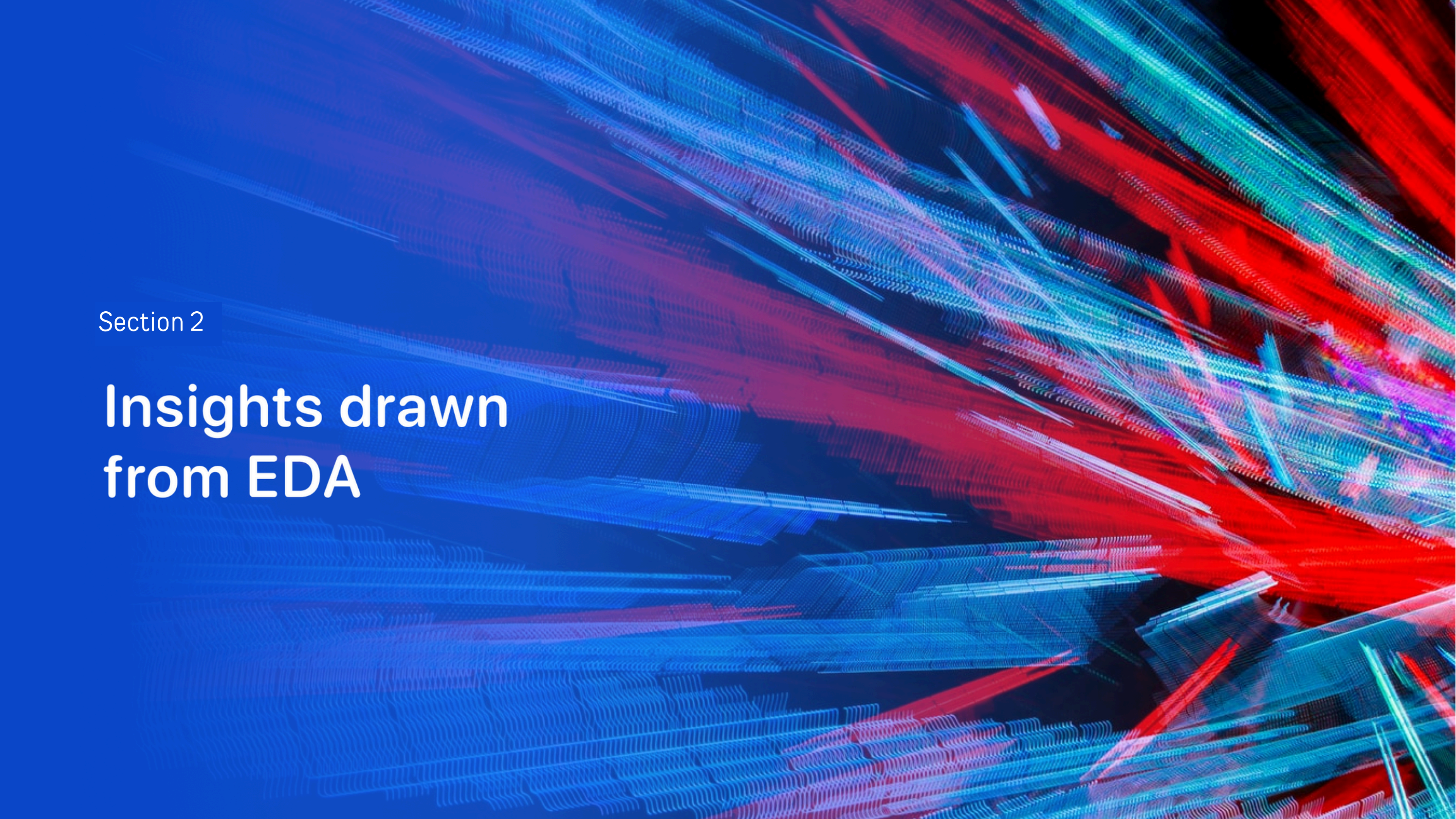
- We built an interactive dashboard with Plotly dash
- We plotted pie charts showing the total launches by a certain sites
- We plotted scatter graph showing the relationship with Outcome and Payload Mass (Kg) for the different booster version.
- Notebook: [Link](#)

Predictive Analysis (Classification)

- We loaded the data using numpy and pandas, transformed the data, split our data into training and testing.
- We built different machine learning models and tune different hyperparameters using GridSearchCV.
- We used accuracy as the metric for our model, improved the model using feature engineering and algorithm tuning.
- We found the best performing classification model.
- Notebook: [Link](#)

Results

- Exploratory data analysis results
- Interactive analytics demo in screenshots
- Predictive analysis results

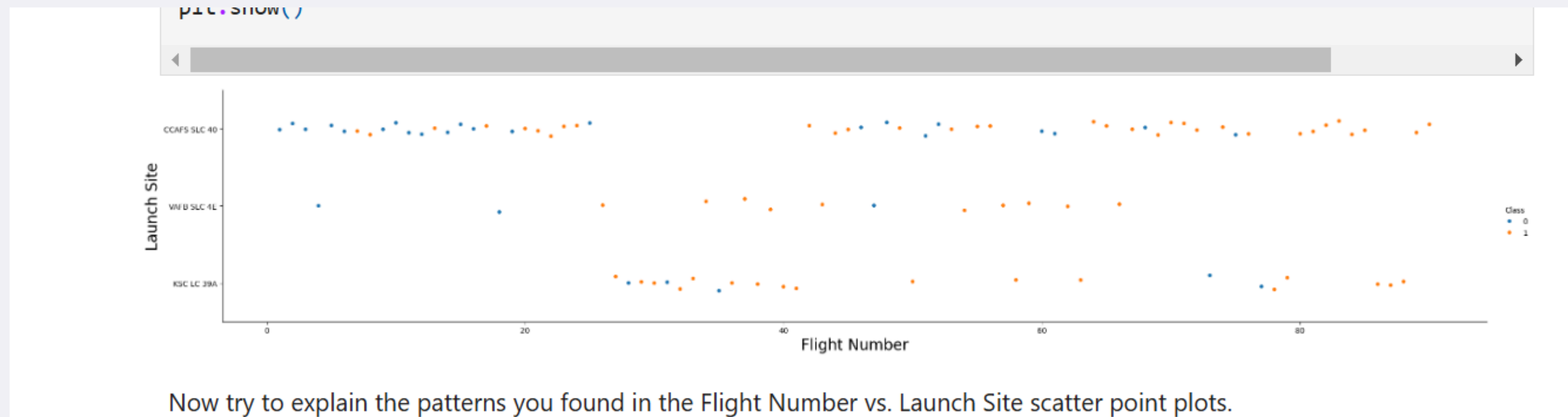
The background of the slide is an abstract composition. It features a dark blue base color. Overlaid on this are numerous diagonal streaks in shades of blue and red, creating a sense of motion or data flow. A faint, light blue grid pattern is also visible, particularly in the lower-left quadrant. The overall effect is high-tech and modern.

Section 2

Insights drawn from EDA

Flight Number vs. Launch Site

- From the plot, we found that the larger the flight amount at a launch site, the greater the success rate at a launch site.

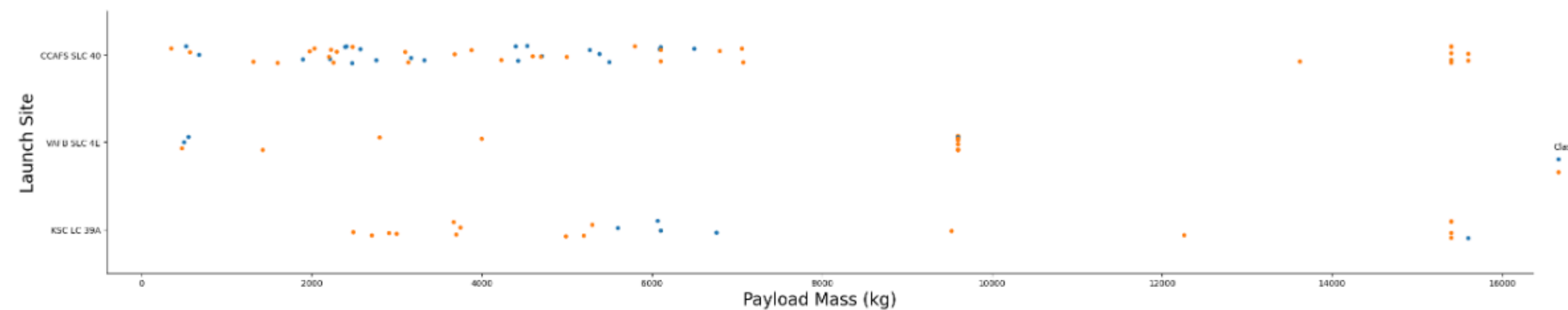


Payload vs. Launch Site

The greater the payload mass for launch site CCAFS SLC 40 the higher the success rate for the rocket.

In [6]:

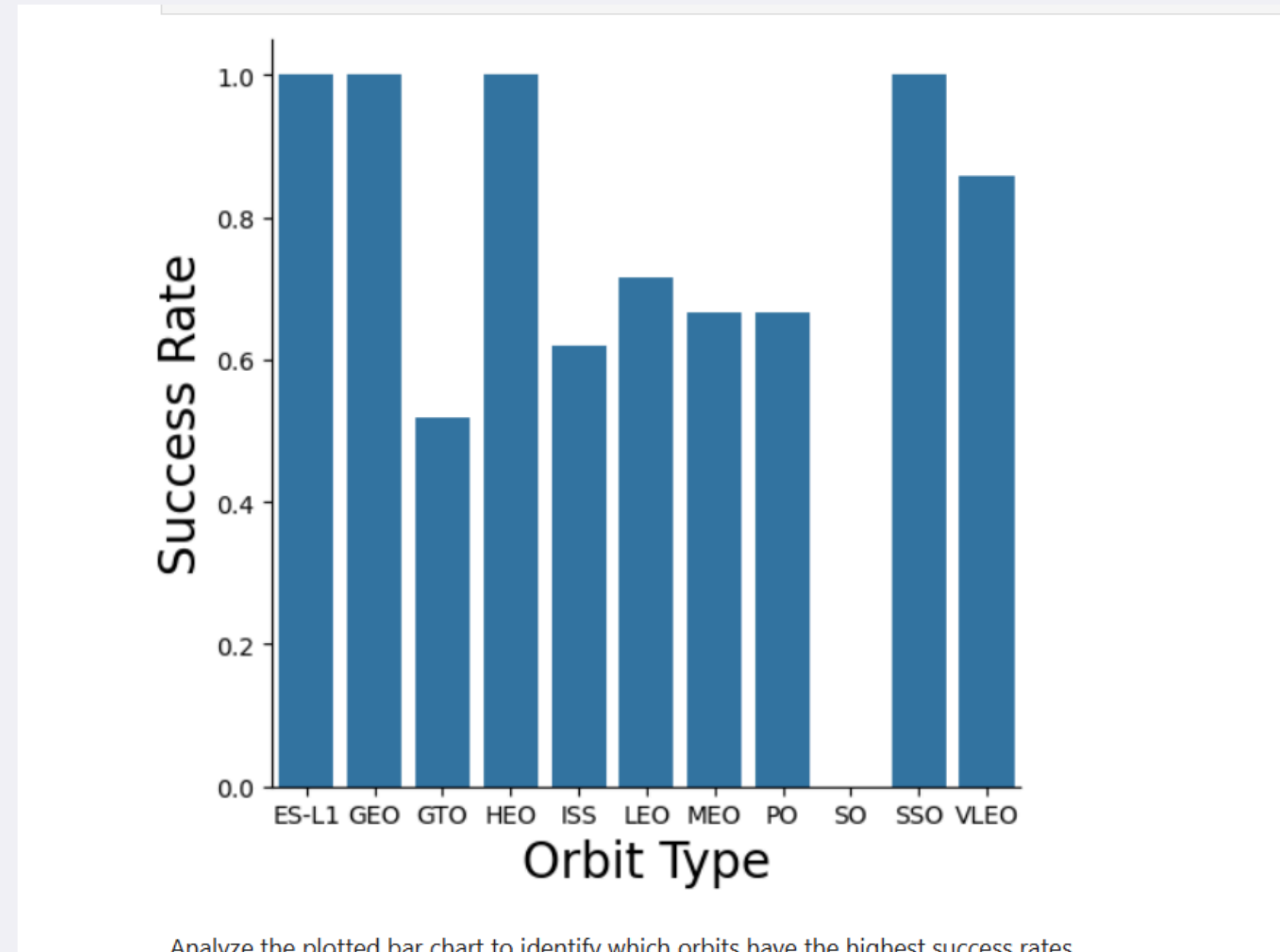
```
# Plot a scatter point chart with x axis to be Pay Load Mass (kg) and y axis to be the Launch site, and hue
sns.catplot(x='PayloadMass', y='LaunchSite', hue='Class', data=df, aspect = 5)
plt.xlabel('Payload Mass (kg)', fontsize=20)
plt.ylabel('Launch Site', fontsize=20)
plt.show()
```



Now if you observe Payload Mass Vs. Launch Site scatter point chart you will find for the VAFB-SLC launchsite there are no rockets launched for heavypayload mass(greater than 10000).

Success Rate vs. Orbit Type

From the plot, we can see that ES-L1, GEO, HEO, SSO, VLEO had the most success rate.



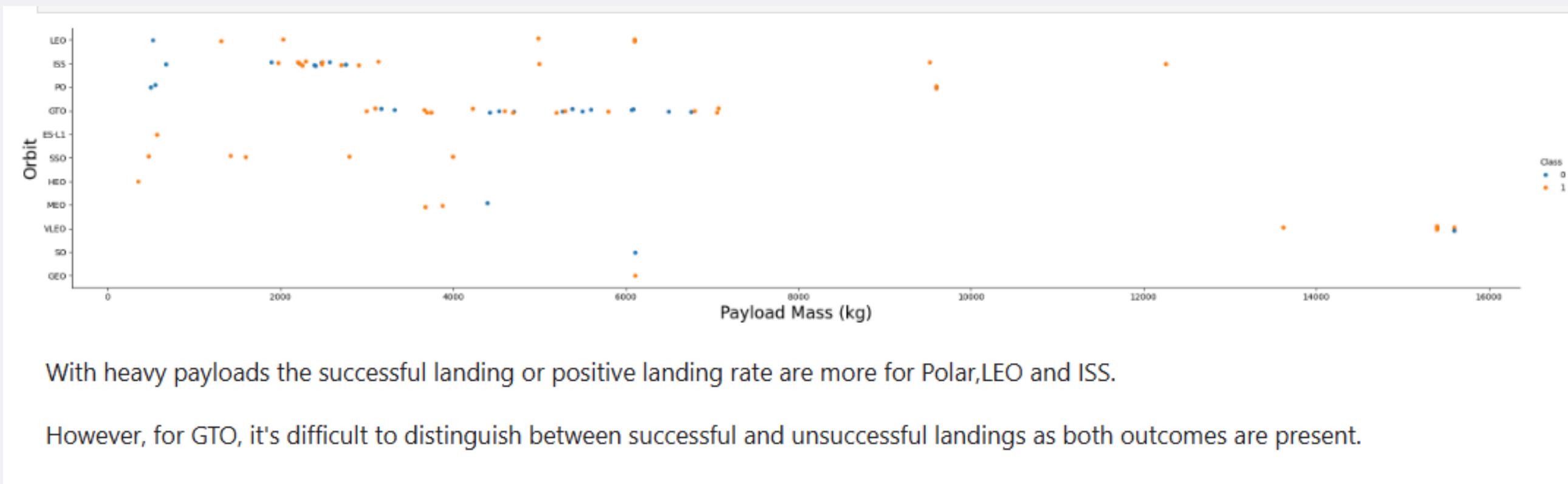
Flight Number vs. Orbit Type

The plot below shows the Flight Number vs. Orbit type. We observe that in the LEO orbit, success is related to the number of flights whereas in the GTO orbit, there is no relationship between flight number and the orbit.



Payload vs. Orbit Type

We can observe that with heavy payloads, the successful landing are more for PO,LEO and ISS orbits.



Launch Success Yearly Trend

From the plot, we can observe that success rate since 2013 kept on increasing till 2020.

All Launch Site Names

We used the key word DISTINCT to show only unique launch sites from the SpaceX data.

Task 1

Display the names of the unique launch sites in the space mission

```
%sql SELECT DISTINCT "Launch_Site" FROM SPACEXTABLE;
```

```
* sqlite:///my_data1.db  
Done.
```

Launch_Site
CCAFS LC-40
VAFB SLC-4E
KSC LC-39A
CCAFS SLC-40

Launch Site Names Begin with 'CCA'

We used the query above to display 5 records where launch sites begin with 'CCA'

Task 2

Display 5 records where launch sites begin with the string 'CCA'

[11]: %sql SELECT * FROM SPACEXTABLE WHERE "Launch_Site" LIKE 'CCA%' LIMIT 5;

* sqlite:///my_data1.db

Done.

[11]:

Date	Time (UTC)	Booster_Version	Launch_Site	Payload	PAYLOAD_MASS_KG	Orbit	Customer	Mission_Outcome	Landing_Outcome
2010-06-04	18:45:00	F9 v1.0 B0003	CCAFS LC-40	Dragon Spacecraft Qualification Unit	0	LEO	SpaceX	Success	Failure (parachute)
2010-12-08	15:43:00	F9 v1.0 B0004	CCAFS LC-40	Dragon demo flight C1, two CubeSats, barrel of Brouere cheese	0	LEO (ISS)	NASA (COTS) NRO	Success	Failure (parachute)
2012-05-22	7:44:00	F9 v1.0 B0005	CCAFS LC-40	Dragon demo flight C2	525	LEO (ISS)	NASA (COTS)	Success	No attempt
2012-10-08	0:35:00	F9 v1.0 B0006	CCAFS LC-40	SpaceX CRS-1	500	LEO (ISS)	NASA (CRS)	Success	No attempt
2013-03-01	15:10:00	F9 v1.0 B0007	CCAFS LC-40	SpaceX CRS-2	677	LEO (ISS)	NASA (CRS)	Success	No attempt

Total Payload Mass

We calculated the total payload carried by boosters from NASA as 45596 using the query below

```
Display the total payload mass carried by boosters launched by NASA (CRS)

[12]: %sql SELECT SUM("PAYLOAD_MASS__KG_") AS total_payload_mass FROM SPACEXTABLE WHERE "Customer" = 'NASA (CRS)';

* sqlite:///my_data1.db
Done.

[12]: total_payload_mass
      45596
```

Average Payload Mass by F9 v1.1

We calculated the average payload mass carried by booster version F9 v1.1 as 2928.4

Task 4

Display average payload mass carried by booster version F9 v1.1

```
[13]: %sql SELECT AVG(PAYLOAD_MASS__KG_) \
      FROM SPACEXTBL \
      WHERE BOOSTER_VERSION = 'F9 v1.1';
```

```
* sqlite:///my_data1.db
```

Done.

```
[13]: AVG(PAYLOAD_MASS__KG_)
      2928.4
```

First Successful Ground Landing Date

We observed that the dates of the first successful landing outcome on ground pad was 22nd December 2015

Task 5

List the date when the first succesful landing outcome in ground pad was acheived.

Hint: Use min function

```
[14]: %sql SELECT MIN("Date") AS first_successful_landing_date FROM SPACEXTABLE WHERE "Landing_Outcome" = 'Success (ground pad)';
```

```
* sqlite:///my_data1.db
```

Done.

```
[14]: first_successful_landing_date
```

```
2015-12-22
```

Successful Drone Ship Landing with Payload between 4000 and 6000

We used the WHERE clause to filter for boosters which have successfully landed on drone ship and applied the AND condition to determine successful landing with payload mass greater than 4000 but less than 6000

Task 6

List the names of the boosters which have success in drone ship and have payload mass greater than 4000 but less than 6000

```
[15]: %sql SELECT DISTINCT "Booster_Version" FROM SPACEXTABLE WHERE "Landing_Outcome" = 'Success (drone ship)' AND "PAYLOAD_MASS_KG_" BETWEEN 4000 AND 6000;
```

```
* sqlite:///my\_data1.db
```

```
Done.
```

```
[15]: Booster_Version
```

```
F9 FT B1022
```

```
F9 FT B1026
```

```
F9 FT B1021.2
```

```
F9 FT B1031.2
```

Total Number of Successful and Failure Mission Outcomes

Calculating the total number of successful and failure mission outcomes

Task 7

List the total number of successful and failure mission outcomes

```
[16]: %sql SELECT "Mission_Outcome", COUNT(*) AS count FROM SPACEXTABLE GROUP BY "Mission_Outcome";
```

```
* sqlite:///my_data1.db
```

```
Done.
```

```
[16]:
```

Mission_Outcome	count
Failure (in flight)	1
Success	98
Success	1
Success (payload status unclear)	1

Boosters Carried Maximum Payload

We determined the booster that have carried the maximum payload using a subquery in the WHERE clause and the MAX()function.

Task 8

List the names of the booster_versions which have carried the maximum payload mass. Use a subquery

```
[17]: %sql SELECT "Booster_Version" FROM SPACEXTABLE WHERE "PAYLOAD_MASS_KG_" = (SELECT MAX("PAYLOAD_MASS_KG_") FROM SPACEXTABLE);
```

```
* sqlite:///my_data1.db  
Done.
```

```
[17]: Booster_Version
```

F9 B5 B1048.4
F9 B5 B1049.4
F9 B5 B1051.3
F9 B5 B1056.4
F9 B5 B1048.5
F9 B5 B1051.4
F9 B5 B1049.5
F9 B5 B1060.2
F9 B5 B1058.3
F9 B5 B1051.6
F9 B5 B1060.3
F9 B5 B1049.7

2015 Launch Records

We used a combination of a WHERE clause and a subquery to filter for the booster version associated with the maximum payload mass. The subquery identifies the maximum value of "PAYLOAD_MASS__KG_", and the main query retrieves the "Booster_Version" corresponding to that value.

Task 9

List the records which will display the month names, failure landing_outcomes in drone ship ,booster versions, launch_site for the months in year 2015.

Note: SQLite does not support monthnames. So you need to use substr(Date, 6,2) as month to get the months and substr(Date,0,5)='2015' for year.

```
[18]: %%sql
      SELECT substr(Date, 1, 4) AS Year, substr(Date, 6, 2) AS Month, Landing_Outcome, Booster_Version, Launch_Site
      FROM SPACEXTABLE
      WHERE substr(Date, 1, 4) = '2015' AND Landing_Outcome = 'Failure (drone ship)';
```

```
* sqlite:///my_data1.db
Done.
```

```
[18]:
```

Year	Month	Landing_Outcome	Booster_Version	Launch_Site
2015	01	Failure (drone ship)	F9 v1.1 B1012	CCAFS LC-40
2015	04	Failure (drone ship)	F9 v1.1 B1015	CCAFS LC-40

Rank Landing Outcomes Between 2010-06-04 and 2017-03-20

We used a combination of the WHERE clause, BETWEEN operator, GROUP BY clause, and ORDER BY clause. The WHERE clause filters data within a specific date range, the GROUP BY clause aggregates the data by Landing_Outcome, and the ORDER BY clause sorts the results by count in descending order.

Task 10

Rank the count of landing outcomes (such as Failure (drone ship) or Success (ground pad)) between the date 2010-06-04 and 2017-03-20, in descending order.

[19]:

```
%%sql
SELECT Landing_Outcome, COUNT(*) AS count
FROM SPACEXTABLE
WHERE Date BETWEEN '2010-06-04' AND '2017-03-20'
GROUP BY Landing_Outcome
ORDER BY count DESC;
```

* sqlite:///my_data1.db

Done.

[19]:

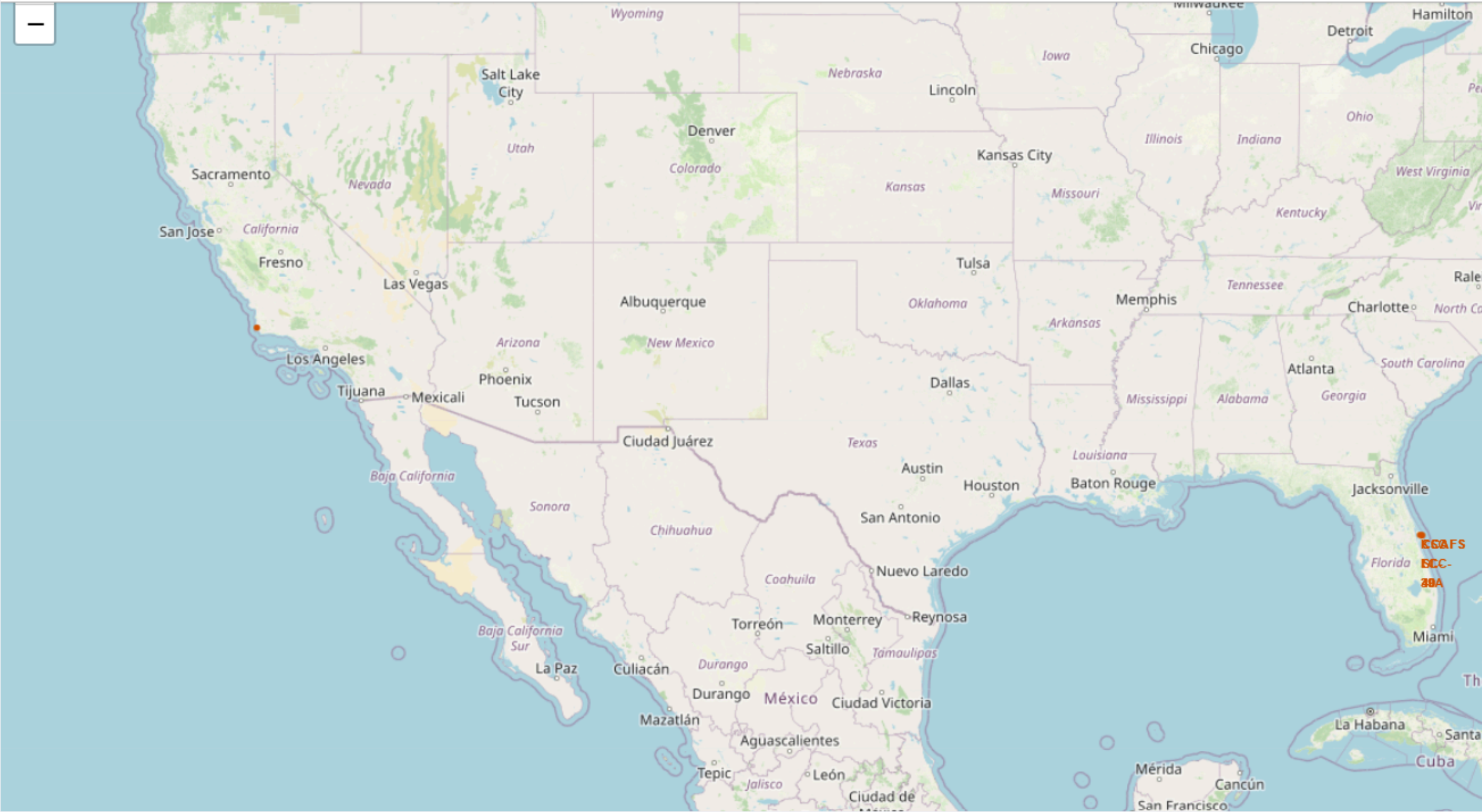
Landing_Outcome	count
No attempt	10
Success (drone ship)	5
Failure (drone ship)	5
Success (ground pad)	3
Controlled (ocean)	3
Uncontrolled (ocean)	2
Failure (parachute)	2
Precluded (drone ship)	1

A satellite view of Earth from space, showing the curvature of the planet and city lights at night. The background is a deep blue space with stars. The Earth's surface is dark blue, with bright yellow and orange lights from cities and towns. The lights are concentrated in the lower right quadrant of the image, forming a dense network of glowing points and lines.

Section 3

Launch Sites Proximities Analysis

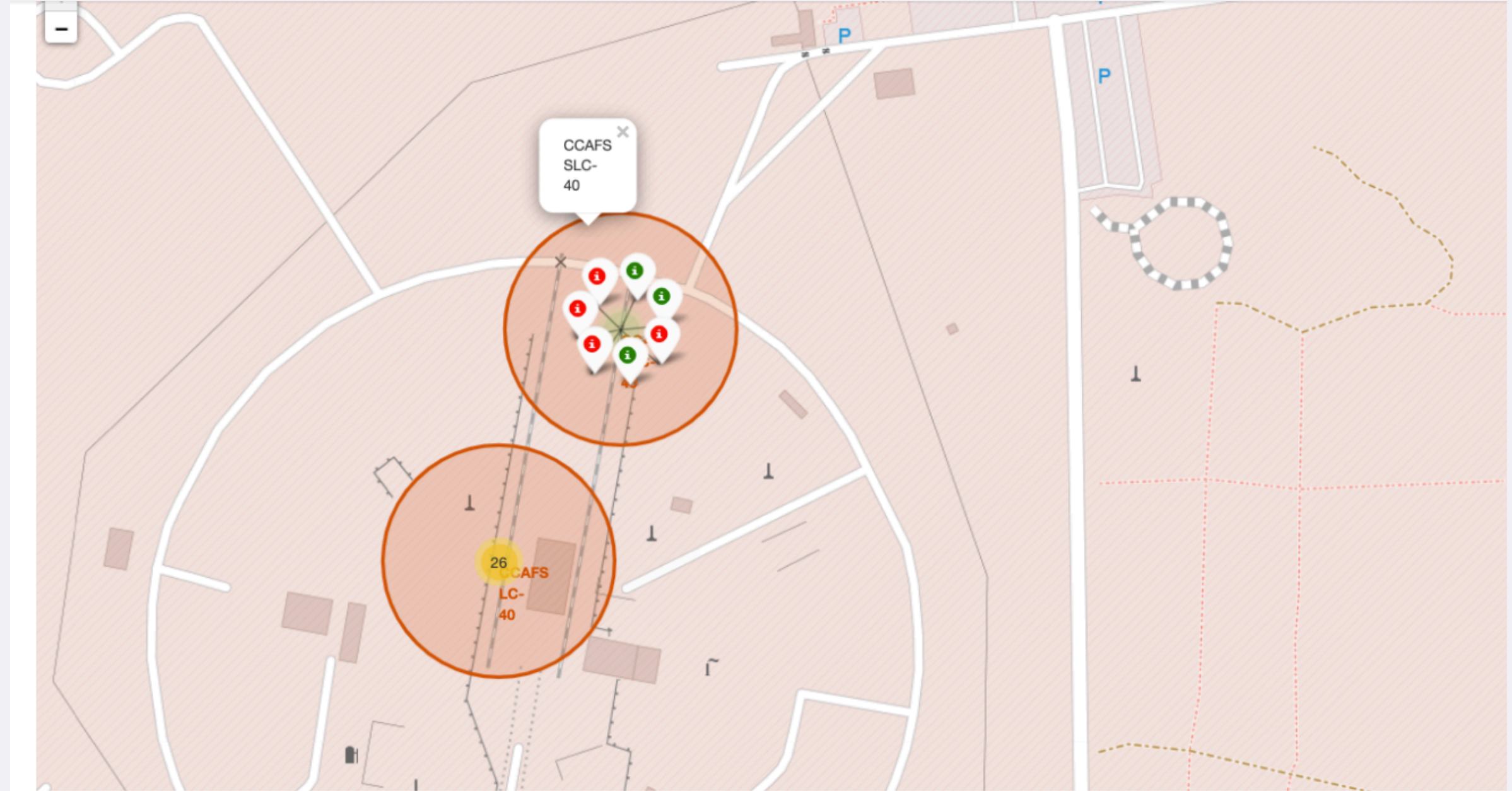
Global Map Markers



Markers showing launch sites with color labels

Green marker showing Successful Launches.

Red marker showing Unsuccessful Launches.



Launch Site Distance to landmarks

Are lunch sites is close proximity to railways?

No

Are lunch sites is close proximity to highway?

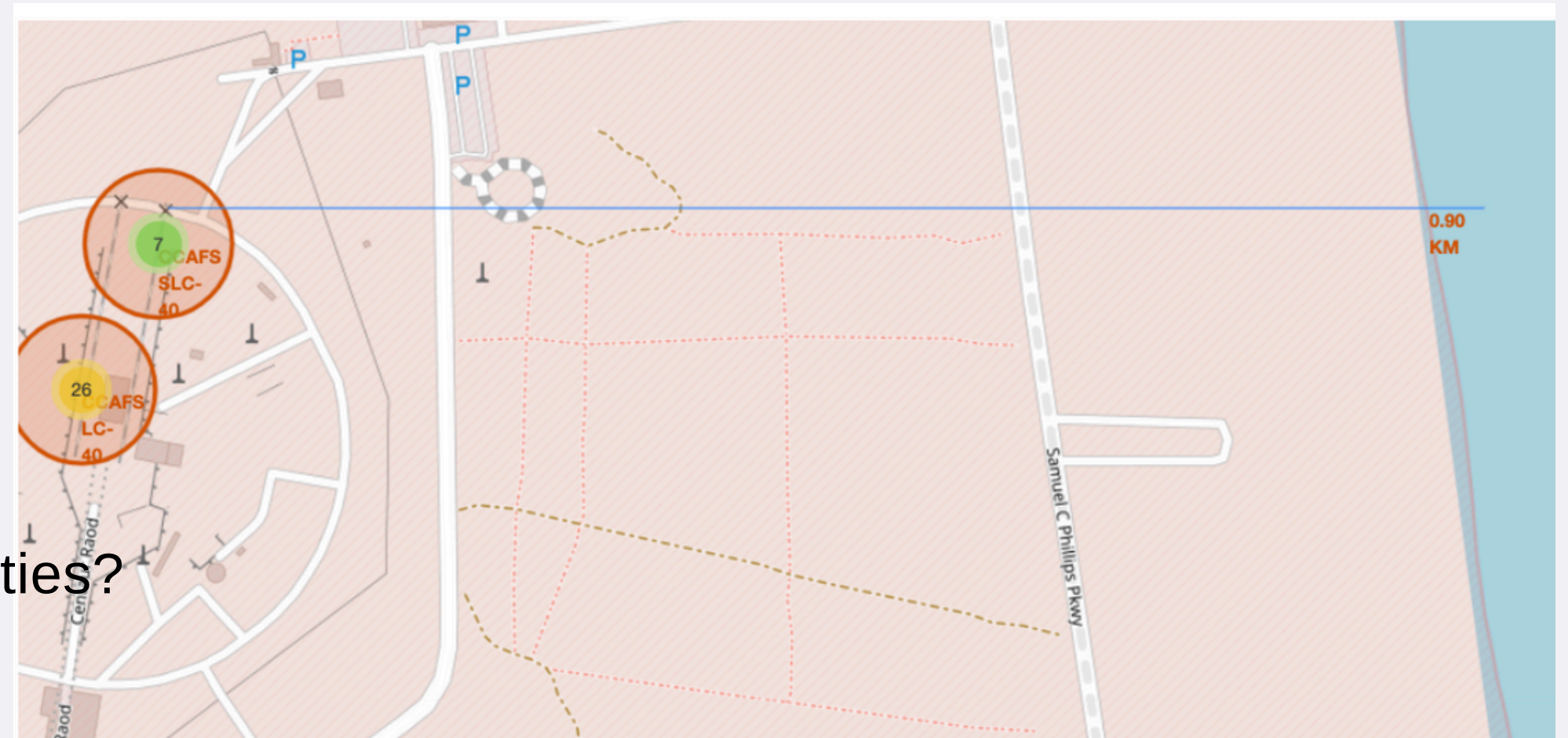
No

Are lunch sites is close proximity to coastline?

Yes

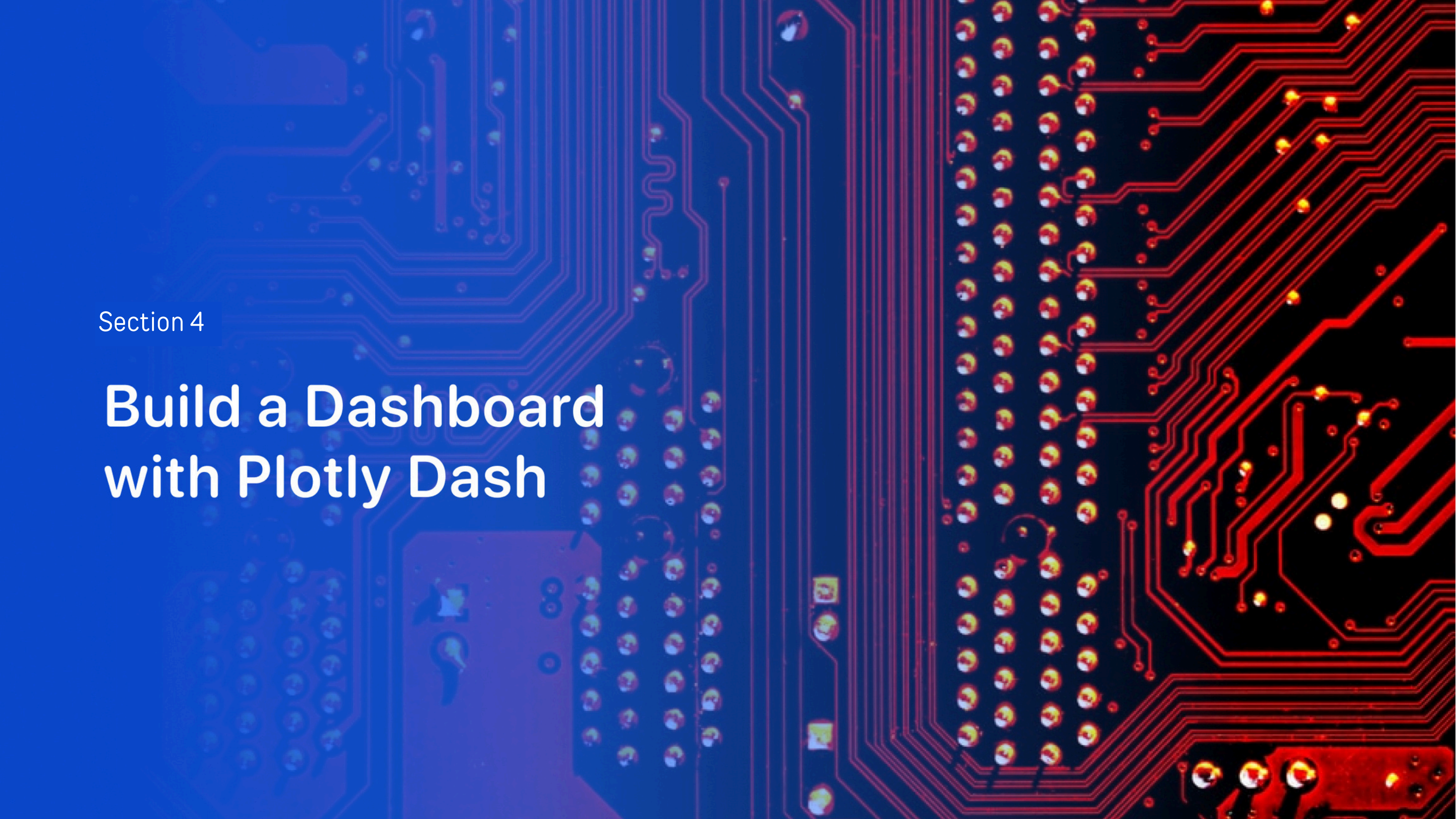
Do launch sites keep certain distance away from cities?

Yes



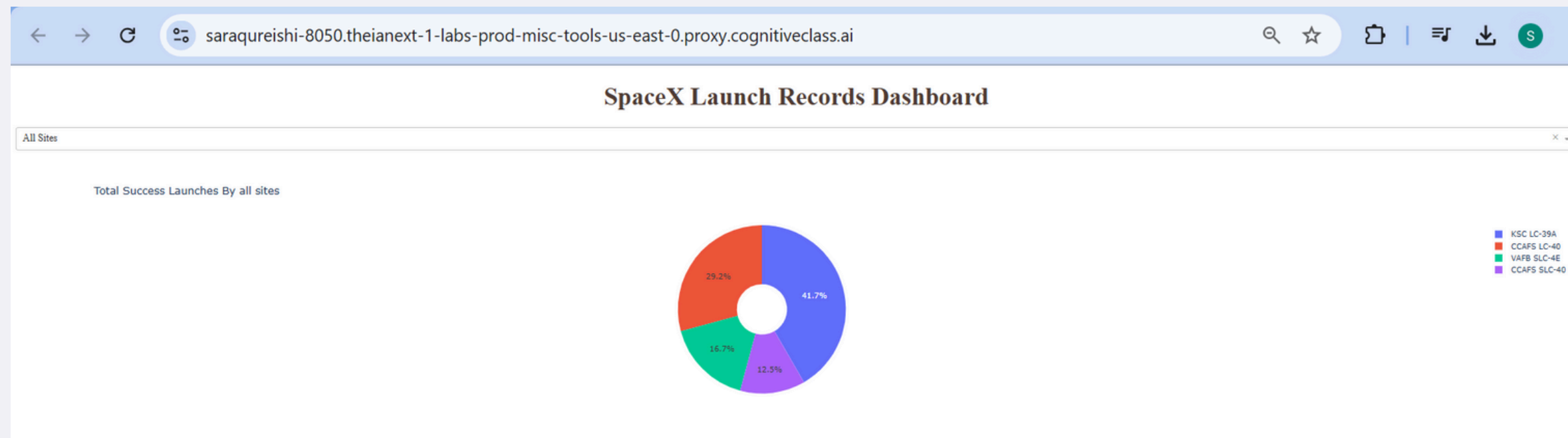
Section 4

Build a Dashboard with Plotly Dash



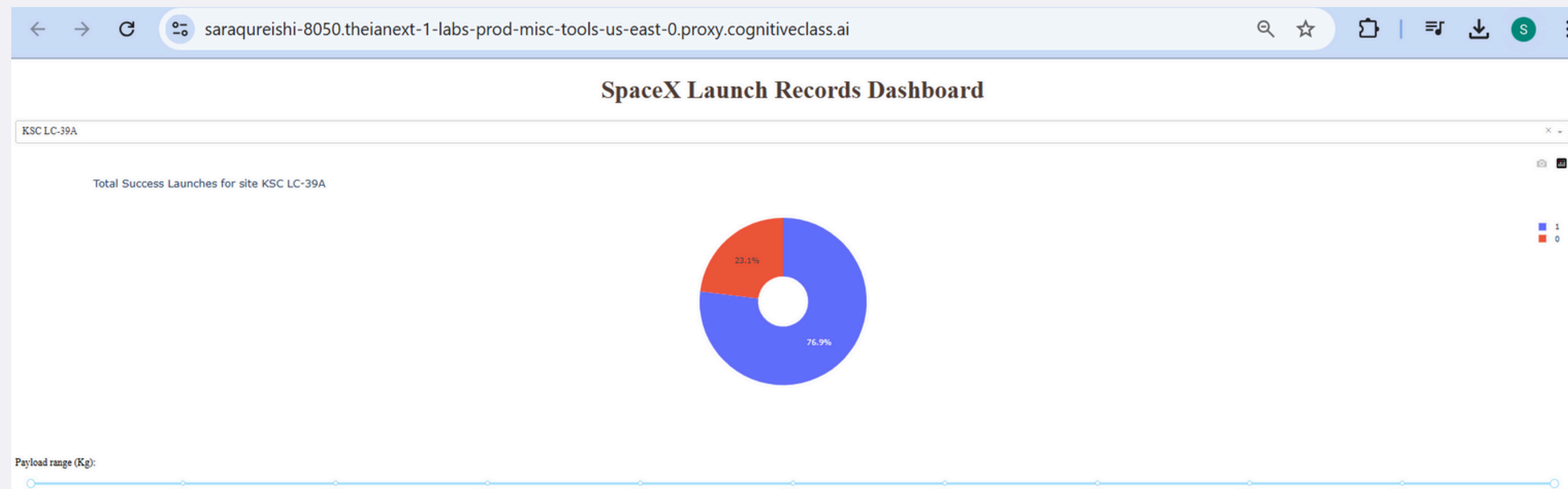
Pie chart showing the success percentage achieved by each launch site

We can see that KSC LC-39A had the most successful launches from all the sites.



Pie chart showing the Launch site with the highest launch success ratio

KSC LC-39A achieved a 76.9% success rate while getting a 23.1% failure rate



Scatter plot of Payload vs Launch Outcome for all sites, with different payload selected in the range slider



Low Weighted Payload 0kg - 4000kg



Heavy Weighted Payload 4000kg - 10000kg

We can see the success rates for low weighted payloads is higher than the heavy weighted payloads.

Section 5

Predictive Analysis (Classification)

Classification Accuracy

The decision tree classifier is the model with the highest classification accuracy

```
In [49]: # Create a dictionary to store the best scores of different models
model_scores = {
    'KNeighbors': knn_cv.best_score_,
    'DecisionTree': tree_cv.best_score_,
    'LogisticRegression': logreg_cv.best_score_,
    'SupportVector': svm_cv.best_score_
}

# Find the model with the highest score
best_model = max(model_scores, key=model_scores.get)

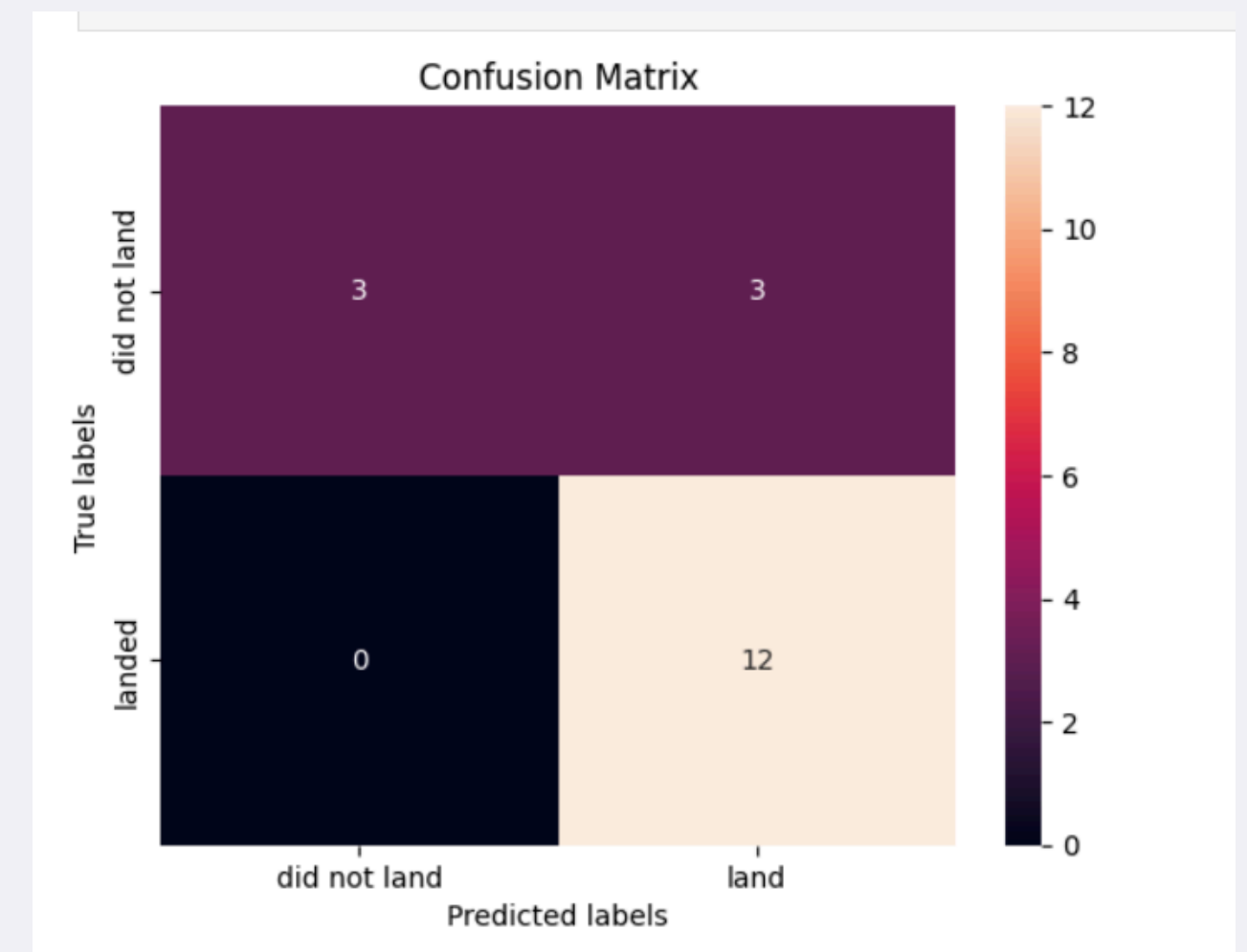
# Print the best model and its score
print(f"The best model is {best_model} with a score of {model_scores[best_model]}")

# Check which model has the best score and print its hyperparameters
if best_model == 'DecisionTree':
    print('Best parameters are:', tree_cv.best_params_)
elif best_model == 'KNeighbors':
    print('Best parameters are:', knn_cv.best_params_)
elif best_model == 'LogisticRegression':
    print('Best parameters are:', logreg_cv.best_params_)
elif best_model == 'SupportVector':
    print('Best parameters are:', svm_cv.best_params_)

The best model is DecisionTree with a score of 0.875
Best parameters are: {'criterion': 'gini', 'max_depth': 2, 'max_features': 'sqrt', 'min_samples_leaf': 1, 'min_samples_split': 2, 'splitter': 'random'}
```

Confusion Matrix

The confusion matrix for the decision tree classifier shows that the classifier can distinguish between the different classes. The major problem is the false positives .i.e., unsuccessful landing marked as successful landing by the classifier.



Conclusions

We can conclude that:

- The larger the flight amount at a launch site, the greater the success rate at a launch site.
- Launch success rate started to increase in 2013 till 2020.
- Orbits ES-L1, GEO, HEO, SSO, VLEO had the most success rate.
- KSC LC-39A had the most successful launches of any sites.
- The Decision tree classifier is the best machine learning algorithm for this task.

Thank you!

